Avaliação de Desempenho — CRUD de Clientes

Oct 15, 2026 · @Dante

Campo	Preencher
Disciplina	*(nome da disciplina)*
Professor(a)	*(nome)*
Integrantes	*(nomes do grupo)*
Repositório	`e-commerce` (API CRUD de clientes)

1. Introdução

A API de clientes sustentou cerca de 4.000 requisições por segundo com 100 usuários simultâneos e nenhum erro, nos três cenários de leitura e escrita. O fator que mais mudou o desempenho não foi a proporção de leituras e escritas nem o tamanho do dataset, e sim a existência de um índice para a consulta executada: uma busca por e-mail sem índice derrubou a vazão para 703 a 1.472 req/s, e o índice em `email` a devolveu ao patamar de 4.000 req/s.

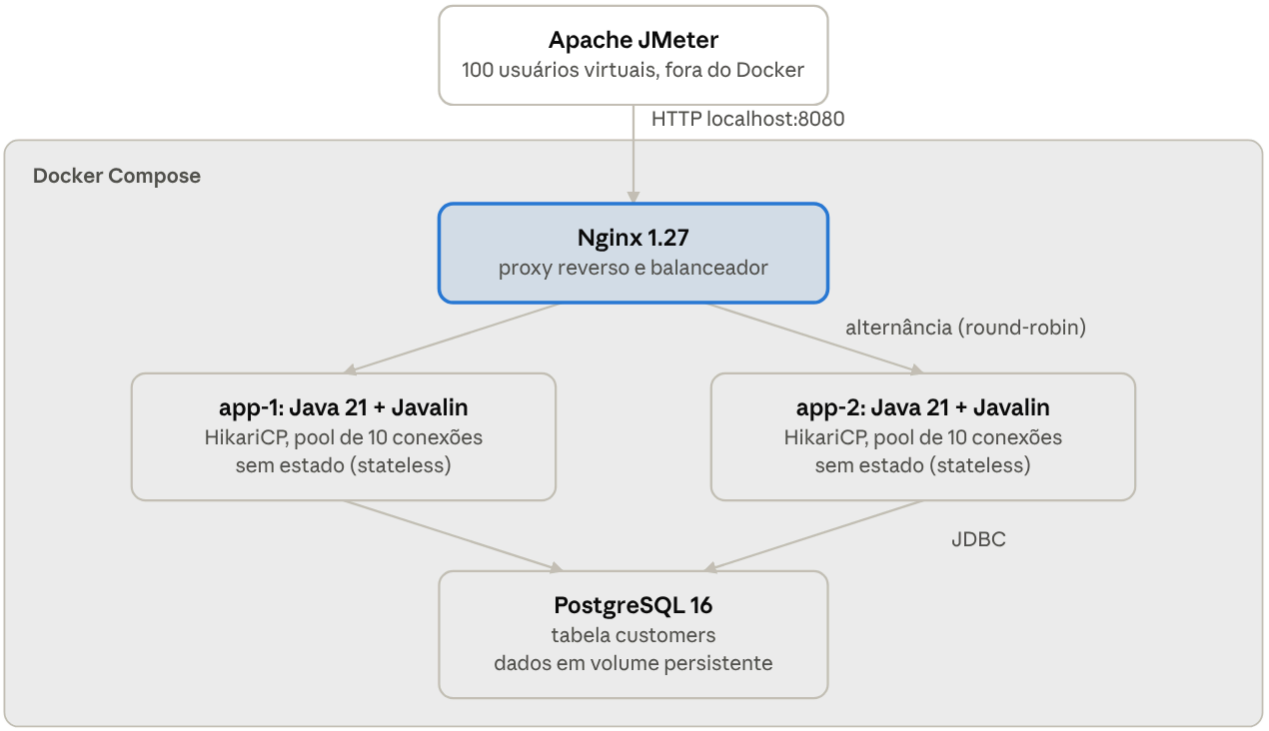
O trabalho implementa uma aplicação CRUD (clientes, servidor web e banco de dados) e avalia seu desempenho em leituras e escritas, respondendo a quatro perguntas:

1. Como a aplicação se comporta nos cenários A (50% leituras / 50% escritas), B (75% / 25%) e C (25% / 75%)?
2. O que acontece se o dataset for reduzido de 50 mil para 20 mil registros?
3. O que acontece com a adição de índices? Qual índice escolher e o desempenho das consultas melhora?
4. Como a escolha entre um banco SQL e um NoSQL impacta o sistema?

2. Parte 1 — Aplicação e arquitetura

A aplicação é uma API REST de cadastro de clientes com as quatro operações CRUD. Toda requisição entra pelo Nginx, que a repassa alternadamente a uma de duas instâncias Java idênticas; as duas compartilham o mesmo banco PostgreSQL.

O Nginx divide a carga entre duas instâncias Java que usam o mesmo PostgreSQL



docker-compose.yml e nginx/nginx.conf do repositório

Como as instâncias não guardam estado, qualquer uma pode atender qualquer requisição, e o gargalo compartilhado passa a ser o banco.

Tecnologias

Camada	Tecnologia	Papel
Servidor web	Java 21 + Javalin 6	Rotas HTTP e regras da API
Acesso ao banco	JDBC + HikariCP	Pool de 10 conexões por instância (20 no total)
Banco de dados	PostgreSQL 16	Armazena a tabela `customers`
Balanceador	Nginx 1.27	Proxy reverso e round-robin entre as instâncias
Infraestrutura	Docker Compose	Sobe os 4 containers
Dataset	Python (gerador próprio)	Gera os clientes com semente fixa
Carga	Apache JMeter 5.6.3	Simula os clientes simultâneos

Modelo de dados

Uma única tabela, `customers`: `id` (BIGSERIAL, chave primária), `name`, `email`, `city`, `age` (18 a 120) e `created_at`. Sem índices extras, o PostgreSQL mantém apenas o índice da chave primária.

Endpoints

Método e rota	Operação	Consulta SQL
`GET /clientes/{id}`	Buscar por ID	`WHERE id = ?`
`GET /clientes?cursor=&limit=`	Listar com paginação por cursor	`WHERE id > ? ORDER BY id LIMIT ?`
`GET /clientes?city=`	Filtrar por cidade	`... AND city = ?`
`GET /clientes?email=`	Buscar por e-mail	`... AND email = ?`
`POST /clientes`	Cadastrar	`INSERT ... RETURNING id`
`PUT /clientes`	Atualizar	`UPDATE ... WHERE id = ?`
`DELETE /clientes/{id}`	Remover	`DELETE ... WHERE id = ?`

A paginação por cursor não executa `COUNT(*)`: busca uma linha a mais que o limite para saber se há próxima página. O filtro por e-mail foi adicionado para o experimento de índices (seção 6).

Dataset

Um gerador em Python (`database/generate_dataset.py`) cria os clientes com semente fixa (20260930): nome sorteado de listas, e-mail único `clienteN@exemplo.test`, uma de 10 capitais e idade entre 18 e 80. A mesma semente gera sempre os mesmos dados, o que torna os experimentos reproduzíveis. A carga no banco usa `COPY`.

3. Cenário dos experimentos

Foram 18 execuções de 60 segundos, cada uma com 100 usuários simultâneos e com o banco recarregado do zero antes de começar.

Ambiente

Item	Valor
Processador	Intel Core i7-13620H (10 núcleos, 16 threads)
Memória	16 GB (8 GB disponíveis para o Docker)
Sistema	Windows 11 + Docker Desktop 29.8 (backend WSL 2)
Gerador de carga	Apache JMeter 5.6.3 em modo não gráfico, na mesma máquina

Cenários

O plano do JMeter (`load-tests/clientes-crud.jmx`) tem dois grupos de usuários que repetem sua operação sem pausa durante 60 segundos, com subida gradual de 5 segundos.

Cenário	Leitores	Escritores	Total
A	50	50	100
B	75	25	100
C	25	75	100

- **Escrita:** `POST /clientes` com nome e e-mail únicos (UUID).
- **Leitura por ID:** `GET /clientes/{id}` com ID sorteado entre 1 e o tamanho do dataset.
- **Leitura por e-mail:** `GET /clientes?email=clienteN@exemplo.test` com N sorteado do mesmo modo; usada no experimento de índices.

Matriz de execuções

Cada combinação abaixo foi executada nos cenários A, B e C, totalizando 18 testes.

Dataset	Leitura	Índice extra
50 mil e 20 mil	por ID	nenhum (só a chave primária)
50 mil e 20 mil	por e-mail	nenhum
50 mil e 20 mil	por e-mail	`idx_customers_email`

Metodologia

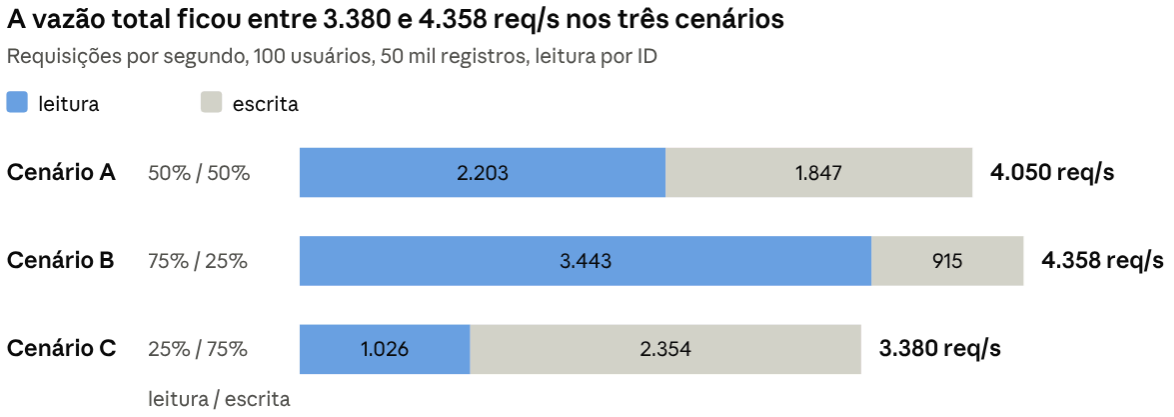
1. Antes de **cada** teste o script `load-tests/run-experiments.sh` esvazia a tabela, recarrega o dataset original, cria ou remove o índice e executa `VACUUM ANALYZE`. Sem isso, os registros inseridos por um teste aumentariam a tabela do próximo (num teste de 60 s os escritores inseriram de 19 mil a 168 mil clientes).
2. Um aquecimento de 20 segundos, descartado, foi executado antes da bateria para que a JVM compilasse o código mais usado.
3. As métricas foram calculadas a partir dos arquivos `.jtl` do JMeter pelo script `relatorio/metricas.py`.
4. Os planos de execução das consultas foram obtidos com `EXPLAIN (ANALYZE, BUFFERS)` pelo script `relatorio/explain.sh`.

Métricas

- **Vazão (throughput):** requisições concluídas por segundo.
- **Tempo de resposta:** média e percentil 95 (p95, o tempo abaixo do qual ficam 95% das requisições), em milissegundos.
- **Erros:** respostas HTTP diferentes de sucesso. Não houve nenhum erro nas 18 execuções (3,23 milhões de requisições).

4. Cenários A, B e C (50 mil registros)

Os três cenários tiveram desempenho semelhante: de 3.380 a 4.358 req/s, tempo médio entre 22 e 28 ms e nenhum erro. A diferença consistente está na operação, não no cenário: a escrita foi mais lenta que a leitura em todas as execuções.



load-tests/results/experiments, execuções 50000-id-idx_none-A/B/C

Cenário	Leitura: média	Leitura: p95	Escrita: média	Escrita: p95
A (50/50)	21,7 ms	28 ms	26,0 ms	33 ms
B (75/25)	20,9 ms	26 ms	26,2 ms	36 ms
C (25/75)	23,2 ms	50 ms	30,3 ms	60 ms

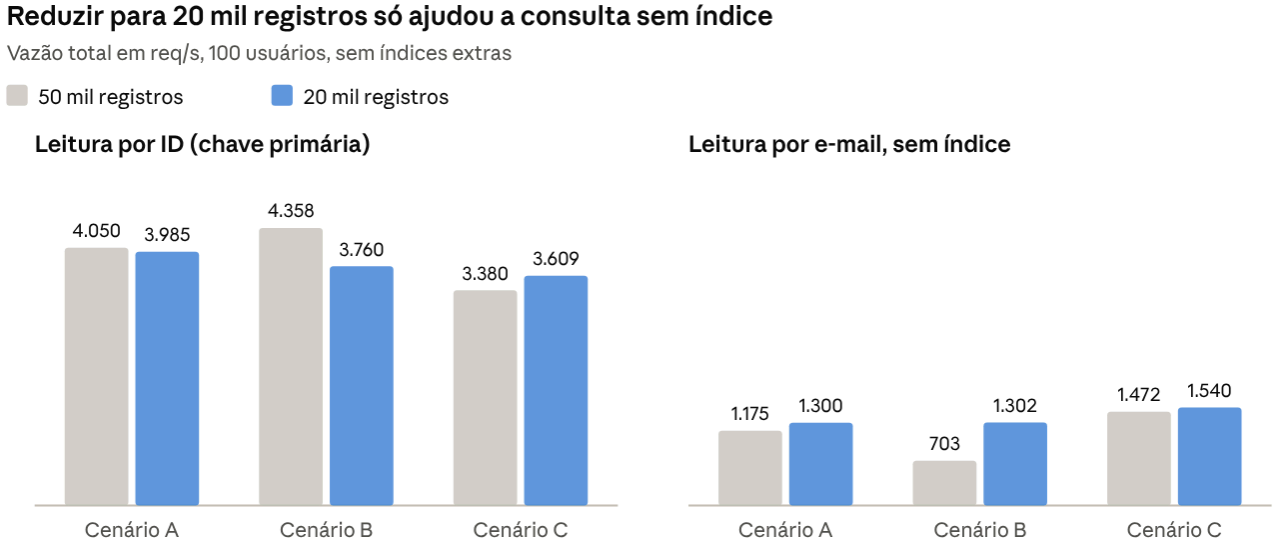
Por que a escrita custa mais. Cada `POST` faz duas consultas: o `INSERT ... RETURNING id` e um `SELECT` para devolver o cliente criado. O `INSERT` ainda grava no log de transações (WAL) e atualiza o índice da chave primária. A leitura por ID é uma única busca nesse índice.

Cenário C. Com 75 escritores, o C teve a menor vazão (3.380 req/s) e o maior p95 (57 ms no total). Todas as inserções disputam o fim da tabela e da árvore do índice, além do WAL. A diferença, porém, é da mesma ordem da variação entre repetições (seção 8), então não deve ser lida como um resultado forte.

100 usuários, 20 conexões. Os 100 usuários compartilham 20 conexões com o banco (10 por instância). Com consultas de poucos milissegundos, as conexões se liberam rápido e não viram gargalo; na seção 6 isso muda.

5. Redução do dataset para 20 mil registros

Reduzir o dataset de 50 mil para 20 mil registros não mudou a leitura por ID, mas melhorou a busca por e-mail sem índice: no cenário B a vazão subiu de 703 para 1.302 req/s e o tempo médio caiu de 136 para 73 ms.



load-tests/results/experiments, execuções *-id-idx_none-* e *-email-idx_none-*

Leitura por ID. A busca pela chave primária percorre uma árvore B+, cujo custo cresce com o logaritmo do número de linhas. De 20 mil para 50 mil a árvore não ganha nem um nível, e as duas tabelas cabem inteiras na memória do PostgreSQL. O `EXPLAIN ANALYZE` mediu 0,12 ms com 20 mil e 0,14 ms com 50 mil. As diferenças de vazão no gráfico (até 14%, ora a favor de um, ora do outro) estão dentro da variação entre execuções.

Leitura por e-mail sem índice. Sem índice, o PostgreSQL lê a tabela inteira (*sequential scan*) para achar um e-mail, e o custo cresce na mesma proporção que o número de linhas: 1,39 ms com 20 mil contra 3,68 ms com 50 mil (2,6 vezes, para 2,5 vezes mais linhas). O efeito é maior no cenário B, que tem mais leitores.

Por que o ganho não foi de 2,5 vezes. Durante o teste os escritores continuam inserindo: nos testes de 20 mil a tabela terminou com 39 mil a 92 mil linhas. Assim, a tabela "pequena" cresce durante a medição e a vantagem diminui ao longo dos 60 segundos.

Resposta. Com o acesso feito por índice, o tamanho do dataset (nesta faixa) quase não importa. Sem índice, o desempenho piora proporcionalmente ao tamanho da tabela, e quanto maior a proporção de leituras, maior o impacto.

6. Índices

O índice escolhido foi `idx_customers_email`, sobre a coluna `email`. Com 50 mil registros, ele levou a busca por e-mail de 703–1.472 req/s para 4.033–4.553 req/s e reduziu o tempo médio de leitura de 83–141 ms para cerca de 20 ms, sem custo mensurável nas escritas.

Qual índice escolher

Um índice só ajuda se o banco o usar numa consulta que a aplicação realmente faz. Por isso cada consulta da API foi medida com `EXPLAIN (ANALYZE, BUFFERS)`, sem índice extra e com cada candidato (50 mil registros):

Consulta	Sem índice extra	Com o índice candidato	O banco usou o índice?
Por ID (`id = ?`)	0,14 ms, chave primária	— (já indexada)	—
Por e-mail (`email = ?`)	3,68 ms, lê a tabela inteira (629 páginas, descarta 49.999 linhas)	0,12 ms com `idx_customers_email` (7 páginas)	Sim: 30 vezes mais rápida

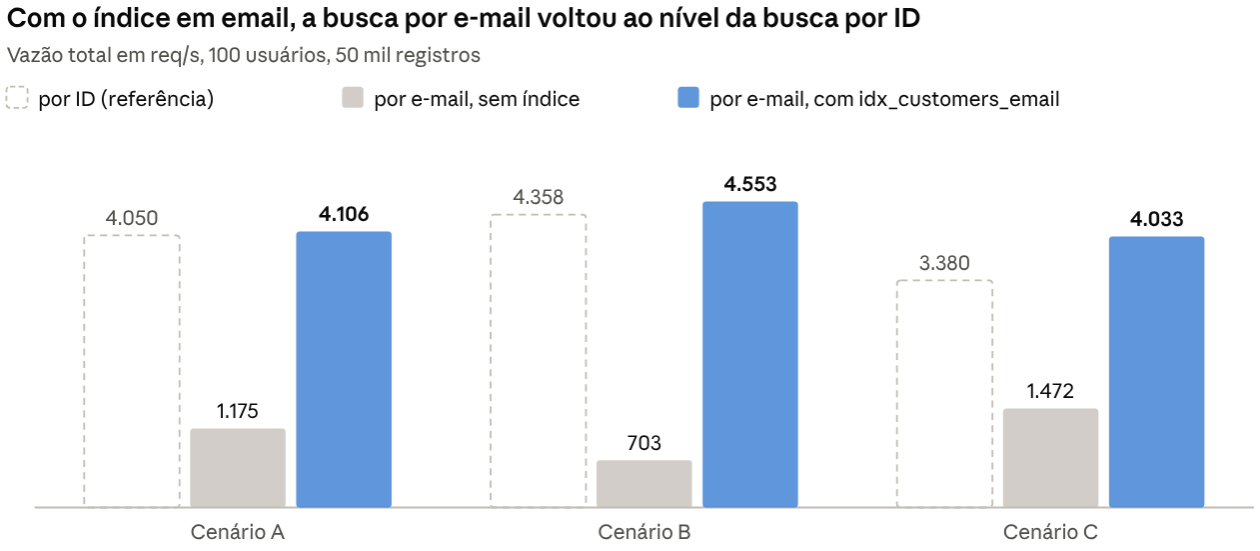
Consulta	Sem índice extra	Com o índice candidato	O banco usou o índice?
Por cidade (`city = ? ORDER BY id LIMIT 21`)	0,08 ms, chave primária + filtro (descarta 183 linhas)	0,10 ms com `(city)` ; 0,09 ms com `(city, id)`	Não, nos dois casos

O índice em `city` , sugerido inicialmente, não ajuda: são só 10 cidades, cada uma com cerca de 10% das linhas (baixa seletividade). Como a consulta pede os 21 primeiros por `id` , o banco percorre a chave primária e encontra 21 clientes da cidade depois de ler uns 200. O índice ocuparia espaço (0,35 MB, ou 1,9 MB na versão `(city, id)`) e seria atualizado a cada inserção sem nunca ser usado.

O e-mail é o oposto: cada valor é único (seletividade máxima) e, sem índice, cada busca lê a tabela inteira. Além disso, buscar cliente por e-mail é uma operação real de sistemas de cadastro (login, verificar duplicidade).

```
CREATE INDEX idx_customers_email ON customers (email);
```

Efeito no desempenho



load-tests/results/experiments, execuções 50000-*

Cenário (50 mil)	Leitura sem índice: média / p95	Leitura com índice: média / p95
A	82,5 ms / 186 ms	20,7 ms / 40 ms
B	140,6 ms / 309 ms	19,9 ms / 24 ms

Cenário (50 mil)	Leitura sem índice: média / p95	Leitura com índice: média / p95
C	82,5 ms / 175 ms	19,6 ms / 38 ms

Com 20 mil registros o efeito foi o mesmo: de 1.300–1.540 req/s sem índice para 3.419–3.564 req/s com índice.

Sem índice, a escrita também piorou. No cenário A sem índice, o `POST` levou 80,6 ms em média, contra 26 ms nos outros testes. As leituras lentas prendem as 20 conexões do pool e a CPU do banco por mais tempo, e as escritas esperam na fila. Uma consulta ruim degrada o sistema inteiro, não só ela mesma.

Custo do índice. Toda inserção passa a atualizar também o índice de e-mail, que ocupa 2,4 MB com 50 mil linhas (a tabela ocupa 4,9 MB). Mesmo no cenário C, com 75 escritores, a escrita com índice (25,6 ms) não ficou mais lenta que sem ele (30,3 ms no teste por ID): o custo existe, mas é menor que a variação entre execuções.

Resposta. Sim, o desempenho das consultas melhora, desde que o índice sirva a uma consulta que o banco não já resolva bem. Um bom índice escolhe-se pela consulta (seletividade e frequência), não pela coluna. Como o e-mail deve ser único por regra de negócio, o ideal seria `CREATE UNIQUE INDEX`, que acelera a busca e impede cadastros duplicados.

7. SQL × NoSQL

Para este sistema, o banco relacional é a escolha adequada. Um banco NoSQL só traria vantagem com um volume de escrita muito maior que o testado, ou com dados sem estrutura fixa, e cobraria por isso em consistência e flexibilidade de consulta. Esta seção é uma análise; não houve experimento com um banco NoSQL.

Aspecto	PostgreSQL (SQL, usado)	Documento (ex.: MongoDB)	Chave-valor ou colunar (ex.: DynamoDB, Cassandra)
Modelo	Tabela com esquema fixo e restrições (`NOT NULL`, `CHECK`, `UNIQUE`)	Documento JSON com esquema flexível	Item acessado por chave de partição
Busca por ID	Índice da chave primária	Índice padrão em `_id`	O acesso mais rápido do modelo

Aspecto	PostgreSQL (SQL, usado)	Documento (ex.: MongoDB)	Chave-valor ou colunar (ex.: DynamoDB, Cassandra)
Busca por e-mail ou cidade	Qualquer coluna, com índice secundário	Igual: sem índice faz *collection scan*	Exige índice secundário global ou tabela extra, planejados antes
Consistência	Transações ACID	ACID em um documento; em vários, com custo	Em geral eventual; forte com custo maior
Escala de escrita	Uma instância; dividir exige particionamento	Sharding nativo	Distribuída por natureza; alta vazão de escrita

O que muda neste sistema.

- **O gargalo é o banco.** As duas instâncias Java não guardam estado; adicionar mais delas não adianta quando o PostgreSQL satura, como na busca sem índice. Um NoSQL distribuído divide a carga entre vários nós e escala melhor as escritas (cenário C). O PostgreSQL escala leituras com réplicas, mas escalar escritas exige particionamento.
- **A lição dos índices vale para os dois.** No MongoDB, uma busca por e-mail sem índice também lê a coleção inteira. Em bancos chave-valor é pior: cada nova forma de consulta (por e-mail, por cidade com paginação) precisa ser planejada no modelo antes, enquanto no SQL basta escrever a consulta e criar um índice.
- **Regras de integridade.** O e-mail único e a idade entre 18 e 120 são garantidos pelo PostgreSQL. Na maioria dos NoSQL essas regras passariam para o código Java, com risco de inconsistência quando duas instâncias gravam ao mesmo tempo.
- **Escala do problema.** O PostgreSQL atendeu 4.000 req/s com 50 mil registros numa única máquina. Um cadastro de clientes desse porte não precisa da escala horizontal do NoSQL; ela compensaria com milhões de clientes e escritas em volume muito maior.

8. Conclusões

1. **Cenários A, B e C.** Com acesso por índice, a aplicação atendeu 100 clientes simultâneos a 3.380–4.358 req/s, cerca de 25 ms por requisição e nenhum erro. A proporção entre leituras e escritas mudou pouco o resultado; a escrita foi um pouco mais cara (26–30 ms contra 21–23 ms da leitura).

2. **Dataset de 20 mil.** Só faz diferença quando a consulta lê a tabela inteira. Nesse caso o custo cai na proporção do tamanho (3,68 ms → 1,39 ms por consulta) e a vazão do cenário B quase dobra. Com índice, 20 mil e 50 mil se comportam igual.
3. **Índices.** O fator de maior impacto do trabalho. O índice em `email` deixou a consulta 30 vezes mais rápida e multiplicou a vazão do sistema por 2,7 a 6,5 (com 50 mil registros). O índice em `city` não seria usado pelo banco. O índice certo depende da consulta, e o `EXPLAIN ANALYZE` mostra se ele é usado.
4. **SQL × NoSQL.** Para um cadastro com regras de integridade e dezenas de milhares de registros, o PostgreSQL basta e garante consistência. NoSQL compensaria com volume de escrita muito maior ou esquema variável.

Limitações

- **Uma execução por configuração.** Uma rodada exploratória anterior, sem recarregar o banco, mediu 4.155 req/s no cenário C, contra 3.380 req/s nesta bateria. Diferenças abaixo de uns 20% não devem ser interpretadas como resultado; as diferenças de 2,3 a 6,5 vezes do experimento de índices estão muito acima disso.
- **JMeter na mesma máquina.** O gerador de carga disputa CPU com os containers, o que limita a vazão máxima medida.
- **Ambiente de desenvolvimento.** Docker Desktop no Windows (WSL 2), com rede local. Os números absolutos não representam um servidor de produção; as comparações entre configurações, sim.
- **A tabela cresce durante o teste.** As inserções aumentam a tabela durante os 60 segundos, o que pesa nos testes sem índice.

Como reproduzir

Com Docker, Python 3 e JMeter instalados, na raiz do repositório:

```
docker compose up -d --build
bash load-tests/run-experiments.sh
python relatorio/metricas.py
bash relatorio/explain.sh
```

O primeiro script leva cerca de 25 minutos e grava os resultados brutos e os relatórios HTML do JMeter em `load-tests/results/experiments/`. O segundo resume as métricas em `relatorio/metricas.csv`, e o terceiro grava os planos de execução em `relatorio/explain/`.